

CE-308 Assignment 2

StreamNest Architecture Modernization Proposal

To: Chief Technology Officer, StreamNest

From: Muqeen Ahmed Soomro, Cloud Architect

Reg no: 2023561

Subject: Google Cloud Platform Migration & Architecture Proof-of-Concept

1. Executive Summary

As StreamNest rapidly scales past 2 million active users, our legacy on-premises infrastructure is creating a severe bottleneck. The current reliance on manual virtual machine deployments and fragmented, unorganized log files forces our engineering and data teams to spend their time managing technical project details rather than focusing on our core mission: understanding our users.

This proposal outlines a modernized, scalable architecture utilizing the Google Cloud Platform (GCP). By migrating to a containerized microservices model and a unified Data Lakehouse, we will eliminate deployment friction. More importantly, this architecture bridges the gap between raw backend events and genuine human understanding, empowering our Data Science team to seamlessly analyze viewing behaviors and drive better content recommendations.

2. Architectural Design & Decisions

A. Backend Modernization: Google Kubernetes Engine (GKE)

- **The Problem:** Our current backend services run on bare virtual machines, suffering from environment inconsistencies that make deployments error-prone and time-consuming.
- **The Solution:** We implemented Docker containerization orchestrated by Google Kubernetes Engine (GKE) Autopilot.
- **The Rationale:** Containerization guarantees that our code behaves identically across development and production environments. By utilizing GKE Autopilot, Google manages the underlying server nodes automatically. This shifts our engineering focus entirely away from hardware maintenance and allows the

platform to automatically scale resources up and down based on real-time user traffic without any downtime.

B. The Data Lakehouse Pipeline: Cloud Storage & BigQuery

- **The Problem:** Raw event logs are currently siloed and difficult to parse, preventing the Data Science team from easily accessing clean data to build recommendation models.
- **The Solution:** A Data Lakehouse architecture utilizing Google Cloud Storage (GCS) as the raw ingestion layer, directly linked to BigQuery as the analytics engine.
- **The Rationale:** GCS acts as a highly durable, low-cost repository for all incoming event logs. By linking this storage to BigQuery via external tables, our data scientists can query massive, raw datasets instantly using standard SQL without needing to move, duplicate, or clean the data first. This setup allows the team to rapidly translate technical log bursts into clear insights about human engagement and regional content performance.

3. Proof of Concept Demonstration

To validate this architecture, my team deployed a working, end-to-end proof of concept within the streamnest-assignment GCP environment.

Phase 1: Scalable Microservice Deployment

We containerized a Python-based "User Activity Logger" API, designed to accept user viewing events. This was deployed to a GKE Autopilot cluster in the us-central1 region. The service successfully exposed a public endpoint and dynamically scaled to process incoming JSON payloads.

```
muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$ kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
activity-logger     LoadBalancer  34.118.235.172   <pending>        80:32531/TCP     15s
kubernetes           ClusterIP      34.118.224.1     <none>           443/TCP          12m
muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$ kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
activity-logger     LoadBalancer  34.118.235.172   <pending>        80:32531/TCP     28s
kubernetes           ClusterIP      34.118.224.1     <none>           443/TCP          12m
muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$ kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
activity-logger     LoadBalancer  34.118.235.172   <pending>        80:32531/TCP     4m12s
kubernetes           ClusterIP      34.118.224.1     <none>           443/TCP          16m
muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$ kubectl get services
NAME                TYPE          CLUSTER-IP      EXTERNAL-IP      PORT(S)          AGE
activity-logger     LoadBalancer  34.118.235.172   35.222.155.22    80:32531/TCP     20m
kubernetes           ClusterIP      34.118.224.1     <none>           443/TCP          33m
muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$

muqeemahmed_co@cloudshell:~/streamnest-api (streamnest-assignment)$ curl -X POST http://35.222.155.22:80 \
-H "Content-Type: application/json" \
-d '{"user_id": "U123", "event": "play", "content_id": "V99"}'
{"logged_event":{"content_id":"V99","event":"play","user_id":"U123"},"status":"success"}
```

Phase 2: Raw Data Ingestion

To simulate real-world streaming data, we generated a batch of user watch events—capturing user IDs, content IDs, watch durations, and device types. This raw CSV payload was successfully ingested into a secure Google Cloud Storage bucket (gs://streamnest-raw-data-streamnest-assignment).

```
muqoemahmed_co@cloudshell:~ (streamnest-assignment)$ gcloud container clusters delete streamnest-cluster --region us-central1 --quiet
Deleting cluster streamnest-cluster...done.
Deleted [https://container.googleapis.com/v1/projects/streamnest-assignment/zones/us-central1/clusters/streamnest-cluster].
muqoemahmed_co@cloudshell:~ (streamnest-assignment)$ gcloud storage rm --recursive gs://streamnest-raw-data-streamnest-assignment
Removing objects:
Removing gs://streamnest-raw-data-streamnest-assignment/watch_events.csv#1777677801028495...
Completed 1/1
Removing buckets:
Removing gs://streamnest-raw-data-streamnest-assignment/...
Completed 1/1
muqoemahmed_co@cloudshell:~ (streamnest-assignment)$
```

Phase 3: BigQuery Analytics Transformation

We established an external table link in BigQuery to the raw data in GCS. We then executed a transformation query to instantly answer a critical business question: *Which content received the most total watch time, and on which devices?*

```
2 CREATE OR REPLACE EXTERNAL TABLE `streamnest-assignment.streamnest_analytics.raw_watch_events` (  
3   user_id STRING,  
4   content_id STRING,  
5   watch_duration_mins INT64,  
6   device_type STRING,  
7   timestamp TIMESTAMP,  
8   region STRING  
9 )  
10 OPTIONS (  
11   format = 'CSV',  
12   uris = ['gs://streamnest-raw-data-streamnest-assignment/watch_events.csv'],  
13   skip_leading_rows = 1  
14 );  
15
```

Query completed

Using on-demand processing quota

All results

Elapsed time	Statements processed	Job status
3 sec	3	SUCCESS

Status	End time	SQL	Action
✔	4:29 AM [2:1]	CREATE OR REPLACE EXTERNAL TABLE `streamnest-assignment.streamnest_analytics.raw_watch	View results
✔	4:29 AM [17:1]	CREATE OR REPLACE TABLE `streamnest-assignment.streamnest_analytics.content_performanc	View results
✔	4:29 AM [31:1]	SELECT * FROM `streamnest-assignment.streamnest_analytics.content_performance`	View results

Query completed

Using on-demand processing quota

←

Query results

Chat

Save results ▾

Open in ▾

↕

Job information

Results

Visualization

JSON

Execution details

Execution graph

Row	content_id ▾	device_type ▾	total_watch_time...	unique_viewers ▾	
1	C202	Laptop	120	1	
2	C101	SmartTV	105	2	
3	C101	Mobile	15	1	
4	C303	Mobile	10	1	

4. Conclusion

This proof-of-concept confirms that GCP can seamlessly support StreamNest’s continued growth. The automated scaling of GKE ensures platform stability during peak streaming hours, while the serverless analytics of BigQuery provides our data teams with immediate access to behavioral data. By adopting this architecture, we remove the technical friction of server management and unlock our ability to build a platform deeply aligned with human understanding and user needs.